

# Relación entre las clases de complejidad P, NP y BQP: límites de la computación clásica y cuántica

Pértile de la Vega Valentina  
valentinapertile@hotmail.com

## Abstract:

Este trabajo analiza la relación entre las clases de complejidad computacional P, NP y BQP, prestando atención a la pregunta de si la computación cuántica puede resolver eficientemente problemas para los cuales solo se conocen algoritmos clásicos de tiempo exponencial. Se explicarán conceptos fundamentales de la teoría de la complejidad, tales como las relaciones conocidas entre las clases mencionadas. Además se examinará por qué el algoritmo de Grover no alcanza para tratar los problemas NP-completos a pesar de tener una aceleración cuadrática. Al marco teórico se lo acompañará de una implementación en Qiskit que demuestra esta limitación mediante una simulación de Grover sobre instancias del problema 3-SAT. Se llega a la conclusión de que la computación cuántica representa un avance cualitativo en ciertas áreas, pero no implica que los problemas NP-completos sean resolubles en tiempo polinomial.

## I. Introducción

Partiendo desde el surgimiento de algoritmos cuánticos en la década de 1990, la relación entre la computación cuántica y los problemas computacionales difíciles plantea una gran expectativa en la informática teórica. La intuición popular suele imaginar que, debido a la superposición cuántica, una computadora cuántica es capaz de probar todas las soluciones al mismo tiempo. Esta idea, aunque es bastante atractiva para el imaginario colectivo, es fundamentalmente incorrecta.

La teoría de la complejidad computacional sirve para estudiar qué problemas se pueden resolver de manera eficiente, tanto en el marco clásico como en el cuántico. La igualdad o no entre las clases P y NP representa uno de los problemas abiertos más famosos de las matemáticas. La incorporación de un modelo cuántico agrega una nueva clase: BQP, que igualmente plantea dudas sobre su relación con las clases clásicas.

## II. Complejidad computacional

La complejidad computacional es el estudio de los recursos de tiempo y espacio que se requieren para resolver problemas computacionales (*Nielsen & Chuang, 2010, p. 139*). La pregunta principal no es si un problema tiene solución, sino cuánto cuesta encontrarla en función del tamaño  $n$  de la entrada. La distinción fundamental es entre problemas tratables, resolubles con recursos polinomiales, y problemas intratables, que requieren recursos exponenciales.

Esta diferencia no es meramente cuantitativa, sino cualitativa. Un algoritmo polinómico que resuelve instancias de tamaño  $n$  en tiempo  $n^3$  se puede ejecutar normalmente. En cambio, un algoritmo exponencial que requiere  $2^n$  operaciones, para  $n=100$ , equivale aproximadamente a  $10^{30}$  operaciones, un número astronómicamente grande que supera cualquier capacidad de cómputo disponible actualmente.

Un principio fundamental que funciona como cimiento de toda la teoría es la tesis de Church-Turing extendida. La misma sostiene que cualquier modelo de computación puede simularse en una máquina de Turing probabilística con a lo sumo un incremento polinomial en operaciones (*Nielsen & Chuang, 2010, p.139*). Si esta tesis fuera correcta, no habría ningún modelo (incluyendo el cuántico) que pueda ofrecer más que una mejora polinomial respecto del modelo clásico.

Utilizando el algoritmo de Shor para factorizar, obtenemos evidencia en contra de esta hipótesis: no se conoce ningún algoritmo clásico eficiente para factorizar, mientras que el cuántico lo resuelve en tiempo polinomial. No obstante, no hay que perder de vista que la inexistencia de tal algoritmo clásico no ha sido demostrada, por lo que el algoritmo de Shor es evidencia fuerte, pero no concluyente.

Una de las principales dificultades del área es que obtener pruebas rigurosas de intratabilidad para determinados problemas es extremadamente difícil (*Nielsen & Chuang, 2010, p. 140*). Dado que el poder computacional de las computadoras cuánticas se relaciona con los problemas abiertos de la complejidad clásica, no puede afirmarse con certeza cuán poderosa es una computadora cuántica.

### **III. Las clases P y NP**

Por un lado, definimos P como el conjunto de todos los problemas de decisión resolubles en tiempo polinomial por una máquina de Turing determinista. En términos más simples, P contiene todos los problemas con solución eficiente conocida. Ejemplos conocidos en la informática son: ordenar una lista  $O(n \log n)$ , o multiplicar números grandes y determinar si un número es primo (algoritmo AKS).

Por otro lado, definimos NP como la clase de problemas cuya solución, una vez propuesta, puede verificarse en tiempo polinomial. Un clásico ejemplo es SAT: dada una fórmula booleana, verificar si una asignación dada la satisface es inmediata, pero asumiendo que P no es igual a NP, encontrar esa asignación desde cero requiere probar una cantidad exponencial de posibilidades en el peor de los casos. Al margen, si P fuera igual a NP existiría un algoritmo polinomial capaz de resolver SAT sin necesidad de recurrir a esa búsqueda por fuerza bruta.

La pregunta P vs NP consiste en determinar si todo lenguaje aceptado por algún algoritmo no determinista en tiempo polinomial también es aceptado por algún algoritmo determinista en tiempo polinomial (*Cook, 1971*). La opinión general es que  $P \neq NP$ , aunque todavía no tiene demostración.

Dentro de NP existe una subclase importante: los problemas NP-completos. Un problema es NP-completo si está en NP y cualquier otro problema en NP puede reducirse a él en tiempo polinomial. Esto lo hace muy difícil debido a que, si existiese un algoritmo polinomial para cualquiera de ellos, todos los demás en NP se podrían resolver eficientemente. Como consecuencia, se probaría que  $P = NP$ . El teorema de Cook-Levin demuestra que SAT es NP-completo (*Cook, 1971*).

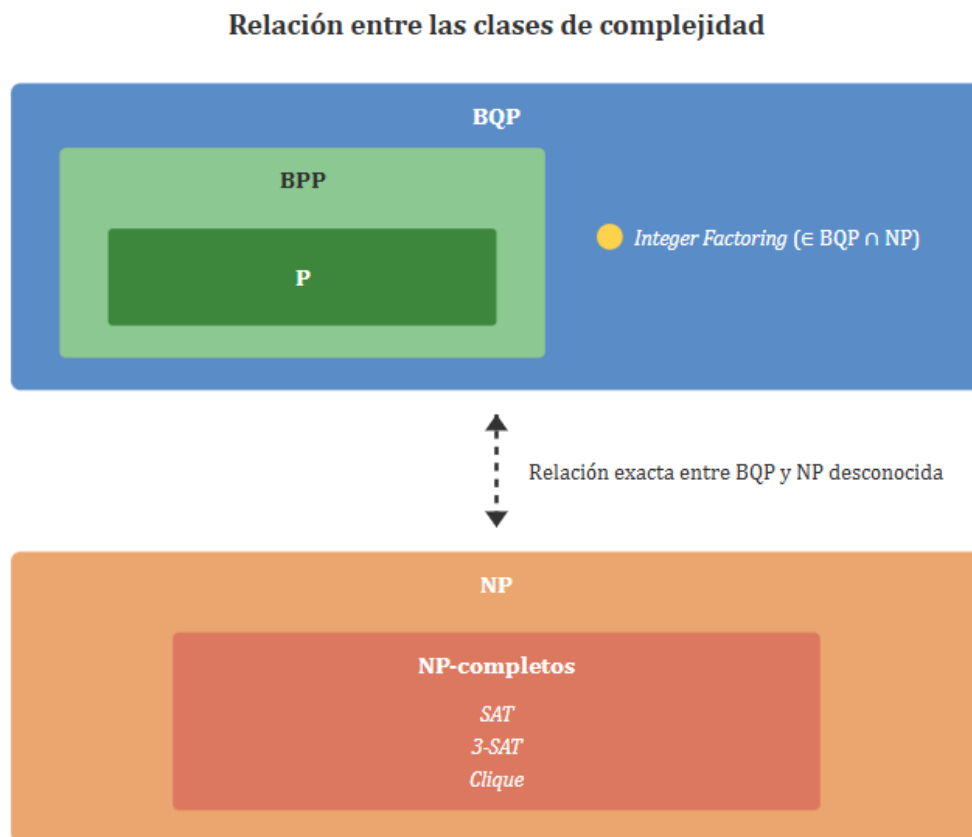
### **IV. La clase BQP y su relación con P y NP**

Antes de introducir BQP, debemos considerar BPP: la clase de problemas resolubles eficientemente con algoritmos aleatorios. BPP engloba a los lenguajes L para los que existe una máquina de Turing probabilística que acepta instancias positivas con probabilidad al menos  $\frac{3}{4}$  y rechaza las negativas con la misma probabilidad (*Nielsen & Chuang, 2010, p.152*). Se la considera la clase de los problemas eficientemente resolubles en el modelo clásico.

Análogamente, existe BQP (tiempo cuántico de error acotado). Es el más interesante en nuestro estudio de algoritmos cuánticos (*Nielsen & Chuang, 2010, p.153*). Un problema está en BQP si existe un circuito cuántico de tamaño polinomial que lo resuelve con probabilidad de error acotada. Se conoce que:  $P \subseteq BPP \subseteq BQP$ , ya que toda computadora cuántica puede simular computadoras clásicas determinísticas y probabilísticas.

La relación entre BQP y NP permanece abierta. Se cree que BQP no está contenido en NP ni tampoco NP en BQP: probablemente se solapan parcialmente (con P y BPP en la intersección), pero ninguna engloba a la otra. Esta conjetura se sostiene, entre otras razones, porque si NP estuviera contenido en BQP, se produciría un colapso de la jerarquía polinomial, algo que la mayoría de los expertos

considera altamente improbable (Aaronson, 2010). Por lo tanto, se conjetura que existen problemas que pueden resolverse eficientemente en BQP, pero que no pertenecen a NP, y viceversa. Esto se visualizará como:



*Nota.* Relaciones conocidas entre las clases de complejidad. Se conoce que  $P \subseteq \text{BPP} \subseteq \text{BQP}$  y que  $P \subseteq \text{NP}$ . La relación exacta entre BQP y NP permanece abierta. La factorización entera pertenece tanto a BQP como a NP, mientras que problemas como *SAT* y *3-SAT* son NP-completos.

La factorización de enteros es un buen ejemplo. Está en NP, verificar factores es inmediato, está en BQP debido al algoritmo de Shor, resoluble en tiempo polinomial cuántico (Shor, 1994). Sin embargo, probablemente no sea NP-completa: ya que es un problema de dificultad intermedia entre P y NP-completo. Si lo fuera, que sea resoluble con Shor implicaría  $\text{NP} \subseteq \text{BQP}$ , algo que los expertos consideran improbable. (Aaronson, 2010).

## V. El algoritmo de Grover y sus límites

El algoritmo de Grover (Grover, 1996) resuelve el problema de la búsqueda no estructurada. Dado un espacio  $N = 2^n$  elementos y una función oráculo que determina si un elemento es la solución buscada, se encuentra la solución en  $O(\sqrt{N})$  consultas, en contraste de  $O(N)$  del mejor algoritmo clásico. Lo hace mediante la amplificación de amplitudes, se aplica iterativamente una inversión de fase (oráculo) y una inversión alrededor de la media. Esto deriva en un aumento de la amplitud del estado de solución en cada iteración. Luego de  $O(\sqrt{N})$  iteraciones, una medición vuelve la solución con alta probabilidad.

Recordando que  $N = 2^n$  es el tamaño del espacio de búsqueda definido más arriba, Grover parece en principio prometedor para atacar problemas NP: si hay  $2^n$  posibles soluciones, Grover las recorre en  $O(\sqrt{2^n}) = O(2^{n/2})$  operaciones. Sin embargo, esta aceleración no convierte el problema en polinomial.

La razón es que  $2^{n/2}$  sigue siendo una función exponencial en  $n$ . Para  $n = 100$  variables, la búsqueda clásica necesita  $2^{100} \approx 10^{30}$  operaciones y Grover la reduce a  $2^{50} \approx 10^{15}$ . Aunque  $10^{15}$  es menor que  $10^{30}$ , sigue siendo inviable.

Además del conteo de iteraciones, no hay que perder de vista el costo de construir el oráculo en sí, que varía dependiendo del problema a analizar y que suele ser no despreciable en muchas aplicaciones prácticas. El análisis anterior cuenta cuántas veces se consulta el oráculo pero no el costo de cada consulta individual. Para una búsqueda no estructurada (podemos tomar como ejemplo una base de datos sin ningún criterio que permita reconocer la solución salvo revisarla una por una), la construcción del oráculo puede necesitar un circuito de tamaño  $O(2^n)$ , lo cual puede anular la ventaja cuántica obtenida por Grover antes de empezar a iterar. La solución cambia para problemas de NP, como 3-SAT: ahí el oráculo se construye a partir del verificador de la solución, que por definición de la clase se puede implementar en tiempo (y por consecuencia tamaño de circuito) polinomial. Esto permite que tenga sentido aplicar Grover a 3-SAT en una primera instancia, pero no elimina el punto: el costo total de la búsqueda cuántica sería la suma del costo de construcción del oráculo más el costo de las  $O(2^{n/2})$  iteraciones. Este primer término suele pasar desapercibido, aunque puede resultar determinante al evaluar la ventaja práctica de Grover.

Más allá de la teoría, esta idea tiene un impacto numérico muy concreto para 3-SAT. El algoritmo aleatorizado de Schöning (Schöning, 1999) resuelve 3-SAT en tiempo  $O(1.334^n)$ , sumado a su refinamiento posterior, conocido como PPSZ (Paturi, Pudlák, Saks & Zane, 1998 con la mejora de Hertli, 2011), lo lleva a  $O(1.308^n)$ . Esto es una opción mejor que el  $O(2^{n/2}) \approx O(1.414^n)$  que ofrece Grover aplicado ingenuamente sobre la búsqueda de fuerza bruta. Es decir, comparado con el mejor algoritmo clásico conocido, y no con la fuerza bruta, Grover por sí mismo pierde.

Sin embargo esto no implica que la técnica detrás de Grover no tenga nada para aportar a 3-SAT. En lugar de aplicar la búsqueda cuántica sobre una búsqueda a ciegas por fuerza bruta, es posible potenciar algoritmos clásicos que ya de por sí son inteligentes mediante una técnica llamada amplificación de amplitud (Brassard, Høyer, Mosca & Tapp, 2000), la cual permite acelerar de forma cuadrática cualquier proceso aleatorio con una probabilidad de éxito conocida. Al utilizar este enfoque sobre el algoritmo aleatorio de Schöning en lugar de una evaluación tradicional, el tiempo de resolución se reduce a  $O(1.153^n)$  (Ambainis, 2004), un rendimiento que supera incluso al refinamiento clásico de PPSZ. Por lo tanto, la clave del beneficio cuántico en los problemas NP-completos no radica en usar a Grover de manera aislada, sino en elegir estratégicamente sobre qué algoritmo clásico se implementará esa aceleración; teniendo siempre en cuenta que, para que esta ganancia sea valiosa, la construcción del oráculo cuántico necesario no debe generar un costo de procesamiento tan alto que termine anulando la ventaja obtenida.

Además, se demostró que cualquier algoritmo cuántico de caja negra para búsqueda no estructurada requiere al menos  $\Omega(\sqrt{N})$  consultas (Bennett et al., 1997). Grover es óptimo ya que no hay ningún algoritmo cuántico que pueda resolver la situación en menos pasos. Esto sirve como techo teórico para afirmar que la ventaja cuántica en búsqueda es cuadrática. Una aceleración cuadrática sobre una base exponencial, al final sigue siendo exponencial.

## VI. Implementación en Qiskit

El siguiente código muestra el contraste entre la búsqueda clásica exhaustiva y el algoritmo de Grover sobre las instancias del problema 3-SAT. Para cada tamaño  $n$  (de 3 a 7 variables), se registra el número de consultas al oráculo y se grafican las curvas en escala logarítmica, junto con la cota teórica

del mejor algoritmo híbrido conocido (Schöningh combinado con amplificación de amplitud), a fin de comparar la ventaja de Grover frente al mejor enfoque clásico disponible.

Cabe aclarar que, a fines didácticos, la solución se determina primero por fuerza bruta para poder construir el circuito que la marca; esto es una simplificación habitual para demostraciones, ya que un oráculo genérico de 3-SAT que verifique la fórmula en superposición requiere una construcción más elaborada. Además, los resultados corresponden a una única instancia aleatoria por cada tamaño  $n$  (porque seleccioné una semilla fija = 42) utilizada para garantizar la reproducibilidad del experimento. Un análisis más robusto promediaría los resultados sobre múltiples instancias por tamaño.

```
import math, numpy as np, matplotlib.pyplot as plt
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator

def generate_3sat(n, seed=42):
    rng = np.random.default_rng(seed)
    return [list(zip(rng.choice(n, 3, replace=False).tolist(),
                    rng.choice([-1, 1], 3).tolist())) for _ in range(n+2)]

def eval_3sat(bits, clauses):
    return all((any((bits[v]==1 and s==1) or (bits[v]==0 and s==-1)
                    for v,s in clause) for clause in clauses))

# Se obtiene una asignación satisfactoria por fuerza bruta únicamente para construir un
# oráculo didáctico. No forma parte del algoritmo de Grover.
def grover_demo(n, clauses):
    sol = next([(i>j)&1 for j in range(n)] for i in range(2**n)
                if eval_3sat([(i>j)&1 for j in range(n)], clauses)), None)
    if sol is None:
        return None

    iters = max(1, round(math.pi/4 * math.sqrt(2**n)))
    qc = QuantumCircuit(n, n)
    qc.h(range(n))

    for _ in range(iters):
        for i,b in enumerate(sol):
            if b==0: qc.x(i)
        qc.h(n-1); qc.mcx(list(range(n-1)), n-1); qc.h(n-1)
        for i,b in enumerate(sol):
            if b==0: qc.x(i)

        qc.h(range(n)); qc.x(range(n))
        qc.h(n-1); qc.mcx(list(range(n-1)), n-1); qc.h(n-1)
        qc.x(range(n)); qc.h(range(n))

    qc.measure(range(n), range(n))
    sim = AerSimulator()
    result = sim.run(qc, shots=1024).result()
    counts = result.get_counts()
    print(counts)
```

```

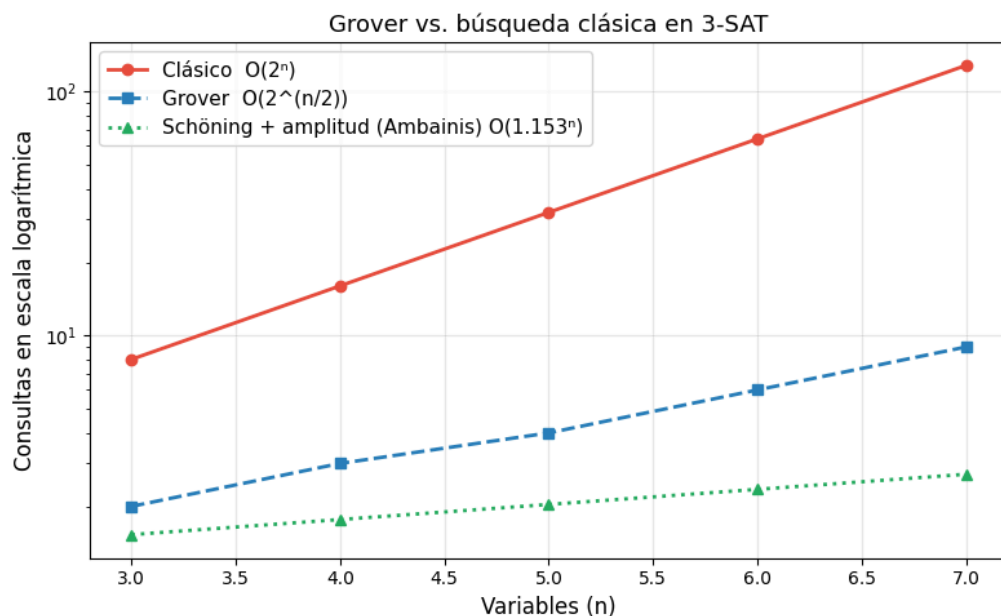
return iters

ns = range(3, 8) # n ≥ 3 porque trabajamos con instancias de 3-SAT
classic, quantum, hybrid = [], [], []

for n in ns:
    classic.append(2**n) #complejidad teórica clásica: búsqueda  $O(2^n)$ 
    clauses = generate_3sat(n)
    iters = grover_demo(n, clauses)
    quantum.append(np.nan if iters is None else iters)
    # Cota del mejor algoritmo híbrido conocido:  $O(1.153^n)$ .
    # No se calcula ejecutando el algoritmo, se grafica como referencia teórica.
    hybrid.append(1.153 ** n)

plt.figure(figsize=(8, 5))
plt.plot(list(ns), classic, 'o-', color='#e74c3c', label='Clásico  $O(2^n)$ ', linewidth=2)
plt.plot(list(ns), quantum, 's--', color='#2980b9', label='Grover  $O(2^{(n/2)})$ ',
linewidth=2)
plt.plot(list(ns), hybrid, '^:', color='#27ae60', label='Schöning + amplitud (Ambainis)  $O(1.153^n)$ ', linewidth=2)
plt.yscale('log') # Escala logarítmica para ver crecimiento exponencial como líneas
plt.xlabel('Variables (n)', fontsize=12)
plt.ylabel('Consultas en escala logarítmica', fontsize=12)
plt.title('Grover vs. búsqueda clásica en 3-SAT', fontsize=13)
plt.legend(fontsize=11); plt.grid(alpha=0.3); plt.tight_layout()
plt.savefig('grover_vs_clasico.png', dpi=150)
plt.show()

```



*Nota. El eje correspondiente a Grover representa el número de iteraciones del algoritmo, no su costo total: cada iteración incluye además una consulta al oráculo cuyo costo, como se discutió en la sección V, puede no ser despreciable. La comparación busca ilustrar el número de pasos de búsqueda y no como una equivalencia de operaciones entre los tres enfoques.*

## **VII. Conclusión**

La computación cuántica no es una solución universal para los problemas computacionalmente difíciles. El análisis de P, NP, BPP y BQP revela una idea: las computadoras cuánticas ofrecen ventajas en dominios muy específicos pero no logran romper la barrera que plantea la clase NP.

El algoritmo de Grover es un ejemplo clave: su aceleración cuadrática es óptima para búsqueda no estructurada, pero su relevancia práctica depende de que el oráculo pueda construirse eficientemente y de la disponibilidad de hardware cuántico tolerante a fallos; en cualquier caso, es insuficiente para hacer polinomiales los problemas NP-completos. Las preguntas permanecen abiertas: no se sabe si  $P = NP$ , no se conoce con exactitud la relación entre BQP y NP, y no existe demostración de que la factorización no esté en P.

Lo que sí es clave es entender que la computación cuántica no rompe la barrera NP, sino que sólo la rodea en ciertos puntos estratégicos. Esto es relevante para considerar las limitaciones de la computación cuántica para evitar tanto el escepticismo injustificado como el entusiasmo desmedido.

## Referencias:

- [1] Nielsen, M. A., & Chuang, I. L. (2010). *Quantum Computation and Quantum Information* (10th anniversary ed.). Cambridge University Press, 139-153.
- [2] Cook, S. A. (1971). The complexity of theorem-proving procedures. *Proceedings of the 3rd Annual ACM Symposium on Theory of Computing*, 151.  
<https://dl.acm.org/doi/epdf/10.1145/800157.805047>
- [3] Shor, P. W. (1994). Algorithms for quantum computation: discrete logarithms and factoring. *Proceedings 35th Annual Symposium on Foundations of Computer Science*, 124-134.  
<https://arxiv.org/abs/quant-ph/9508027>
- [4] Grover, L. K. (1996). A fast quantum mechanical algorithm for database search. *Proceedings of the 28th Annual ACM Symposium on Theory of Computing*, 1-2.  
<https://arxiv.org/abs/quant-ph/9605043>
- [5] Bennett, C. H., Bernstein, E., Brassard, G., & Vazirani, U. (1997). Strengths and weaknesses of quantum computing. *SIAM Journal on Computing*, 1-3.  
<https://arxiv.org/abs/quant-ph/9701001>
- [6] Aaronson, S. (2010). BQP and the polynomial hierarchy. *Proceedings of the 42nd ACM Symposium on Theory of Computing*, 2.  
<https://arxiv.org/abs/0910.4698>
- [7] Schöning, U. (1999). A probabilistic algorithm for k-SAT and constraint satisfaction problems. *Proceedings of the 40th Annual Symposium on Foundations of Computer Science*, 410-414.  
<https://ieeexplore.ieee.org/document/814612>
- [8] Hertli, T. (2011). 3-SAT faster and simpler: Unique-SAT bounds for PPSZ hold in general.  
<https://arxiv.org/abs/1103.2165>
- [9] Brassard, G., Høyer, P., Mosca, M., & Tapp, A. (2000). *Quantum Amplitude Amplification and Estimation*.  
<https://arxiv.org/abs/quant-ph/0005055>
- [10] Ambainis, A. (2004). *Quantum search algorithms*.  
<https://arxiv.org/abs/quant-ph/0504012>